SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART II

ALGORITHM ENGINEERING: IMPLEMENT, TEST, DEBUG & OPTIMISE

Assignment Details	Information
Course	Design and Analysis of Algorithms
Course Outcome	CO4 – Solve optimization problems using Dynamic Programming and Greedy strategies
Nature of Assignment	Individual
Duration	One Week
Total Marks	50
Submission Date	04-11-26
Student Name	G. Sirisha
Register Number	192472170

Task 1 – Build and Validate a Dynamic Programming Solution

Problem: Word Wrap

1.1 Algorithm and DP formulation

Input: A sequence of words and maximum line width \(W\).

Output: Minimum penalty, line arrangement, unused spaces and individual line penalties.

The penalty for a non-final line is:

$$Penalty = (Unused\ Spaces)^2$$

The final line has zero penalty.

Define:

$$DP[i] = \text{minimum penalty for arranging words from } i \text{ to } n$$

Recurrence:

$$DP[i] = \min_{j \geq i}\{Cost(i,j) + DP[j+1]\}$$

Where:

$$Cost(i,j) = \begin{cases} (W-L)^2 & j < n \\ 0 & j = n \end{cases}$$

Here, (L) is the total length of the words and spaces in the current line.

1.2 Intermediate DP states

Use the input:

Dynamic programming creates optimal document layouts efficiently

The program displays:

- DP suffix costs.
- Next-break indices.
- Selected words on every line.
- Unused spaces and penalties.

For example, at width 20:

DP state	**Minimum penalty**
DP[0]	42
DP[1]	17
DP[2]	41
DP[3]	16
DP[4]	16
DP[5]	0

Interpretation: DP[0] = 42 represents the minimum possible penalty for arranging the entire document. Each subsequent state represents the minimum remaining cost after earlier words have been placed.

1.3 Requirement change – Predictions and results

Width	Prediction before execution	Actual minimum penalty
16	Smaller width; more lines and higher penalty	107
20	First change expected in the first line	42
24	More words should fit together; lower penalty	25

Comparison:

- Width 16: Dynamic is placed on the first line.
- Width 20: Dynamic programming fits on the first line.
- Width 24: Dynamic programming still fits, but document also moves into the second line.

The actual arrangement changes as predicted, and the minimum penalty decreases as the width increases.

1.4 Boundary test

Input:

Words: ALGORITHM DP WORKS

Width: 9

Expected behaviour:

- A word exactly equal to the width must fit.
- No negative unused-space value should occur.
- The algorithm must not enter an infinite loop.

The program produces a valid arrangement with zero total penalty.

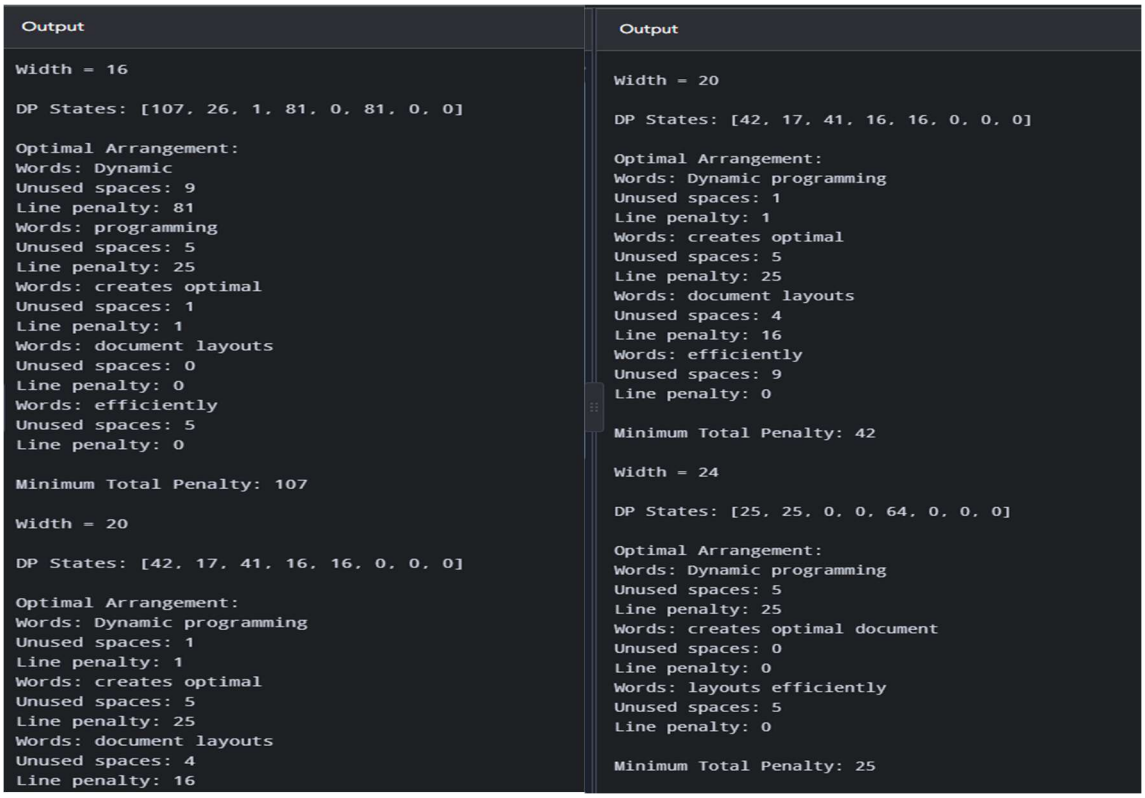
Interpretation: This test checks exact-width handling and prevents buffer/indexing and line-fitting errors.

Python Implementation

```
def word_wrap(words, width):
    n = len(words)
    if width <= 0:
```

```
    raise ValueError("Width must be positive")
if any(len(word) > width for word in words):
    raise ValueError("Word exceeds line width")
dp = [float('inf')] * (n + 1)
split = [-1] * (n + 1)
dp[n] = 0
for i in range(n - 1, -1, -1):
    length = 0
    for j in range(i, n):
        length += len(words[j]) + (1 if j > i else 0)
        if length > width:
            break
        spaces = width - length
        if j == n - 1:
            penalty = 0
        else:
            penalty = spaces ** 2
        cost = penalty + dp[j + 1]
        if cost < dp[i]:
            dp[i] = cost
            split[i] = j + 1
print("\nDP States:", dp)
print("\nOptimal Arrangement:")
i = 0
while i < n:
    j = split[i]
    line = words[i:j]
```

```
        used = sum(len(w) for w in line) + len(line) - 1
        unused = width - used
        penalty = 0 if j == n else unused ** 2
        print("Words:", " ".join(line))
        print("Unused spaces:", unused)
        print("Line penalty:", penalty)
        i = j
    print("\nMinimum Total Penalty:", dp[0])
words = "Dynamic programming creates optimal document layouts efficiently".split()
print("Width = 16")
word_wrap(words, 16)
print("\nWidth = 20")
word_wrap(words, 20)
print("\nWidth = 24")
word_wrap(words, 24)
```



```
Output

Width = 16

DP States: [107, 26, 1, 81, 0, 81, 0, 0]

Optimal Arrangement:
Words: Dynamic
Unused spaces: 9
Line penalty: 81
Words: programming
Unused spaces: 5
Line penalty: 25
Words: creates optimal
Unused spaces: 1
Line penalty: 1
Words: document layouts
Unused spaces: 0
Line penalty: 0
Words: efficiently
Unused spaces: 5
Line penalty: 0

Minimum Total Penalty: 107

Width = 20

DP States: [42, 17, 41, 16, 16, 0, 0, 0]

Optimal Arrangement:
Words: Dynamic programming
Unused spaces: 1
Line penalty: 1
Words: creates optimal
Unused spaces: 5
Line penalty: 25
Words: document layouts
Unused spaces: 4
Line penalty: 16
```

```
Output

Width = 20

DP States: [42, 17, 41, 16, 16, 0, 0, 0]

Optimal Arrangement:
Words: Dynamic programming
Unused spaces: 1
Line penalty: 1
Words: creates optimal
Unused spaces: 5
Line penalty: 25
Words: document layouts
Unused spaces: 4
Line penalty: 16
Words: efficiently
Unused spaces: 9
Line penalty: 0

Minimum Total Penalty: 42

Width = 24

DP States: [25, 25, 0, 0, 64, 0, 0, 0]

Optimal Arrangement:
Words: Dynamic programming
Unused spaces: 5
Line penalty: 25
Words: creates optimal document
Unused spaces: 0
Line penalty: 0
Words: layouts efficiently
Unused spaces: 5
Line penalty: 0

Minimum Total Penalty: 25
```

Task 2 – Greedy vs Dynamic Programming

Problem: Fractional Knapsack and 0/1 Knapsack

2.1 Normal case

Consider the following dataset.

Item	Weight	Value	Value/Weight
A	10	60	6
B	20	100	5
C	30	120	4

Knapsack capacity = 50.

Fractional Knapsack – Greedy decision sequence

1. Select all of A: weight 10, value 60.
2. Select all of B: weight 20, value 100.
3. Select (20/30) of C: value 80.

Total value:

$$60 + 100 + 80 = \boxed{240}$$

For comparison, the 0/1 DP solution selects B and C:

$$100 + 120 = \boxed{220}$$

2.2 Break the Greedy Intuition

Construct the following counterexample.

Item	Weight	Value	Value/Weight
A	10	100	10
B	20	180	9
C	20	160	8

Capacity = 40.

Greedy decision:

- Select A first because it has the highest ratio.
- Select B next.

- Remaining capacity = 10.
- C cannot fit.

Greedy result:

$$100 + 180 = \boxed{280}$$

DP decision:

Select B and C:

$$180 + 160 = \boxed{340}$$

First greedy commitment that prevents the optimum: Selecting A consumes 10 units of capacity. This prevents selecting both B and C, which together provide a higher value.

2.3 Engineering comparison

Parameter	**Fractional Greedy**	**0/1 DP**
Strategy	Value/weight sorting	DP table
Time complexity	(O(n log n))	(O(nW))
Memory	(O(n))	(O(nW))
Selection	Whole or fractional items	Whole items only
Optimality	Optimal for fractional case	Optimal for 0/1 case
Computation	Less work	More work

The implementations demonstrate that greedy selection is effective when fractions are allowed, but it does not guarantee an optimal 0/1 solution.

2.4 Changed requirement

Management requires a very quick, near-optimal answer.

Conclusion: I would select the greedy strategy for a quick initial 0/1 solution when approximate quality is acceptable. The counterexample shows that this can sacrifice value (280 instead of 340). If the exact result is essential, DP is preferable when capacity is manageable.

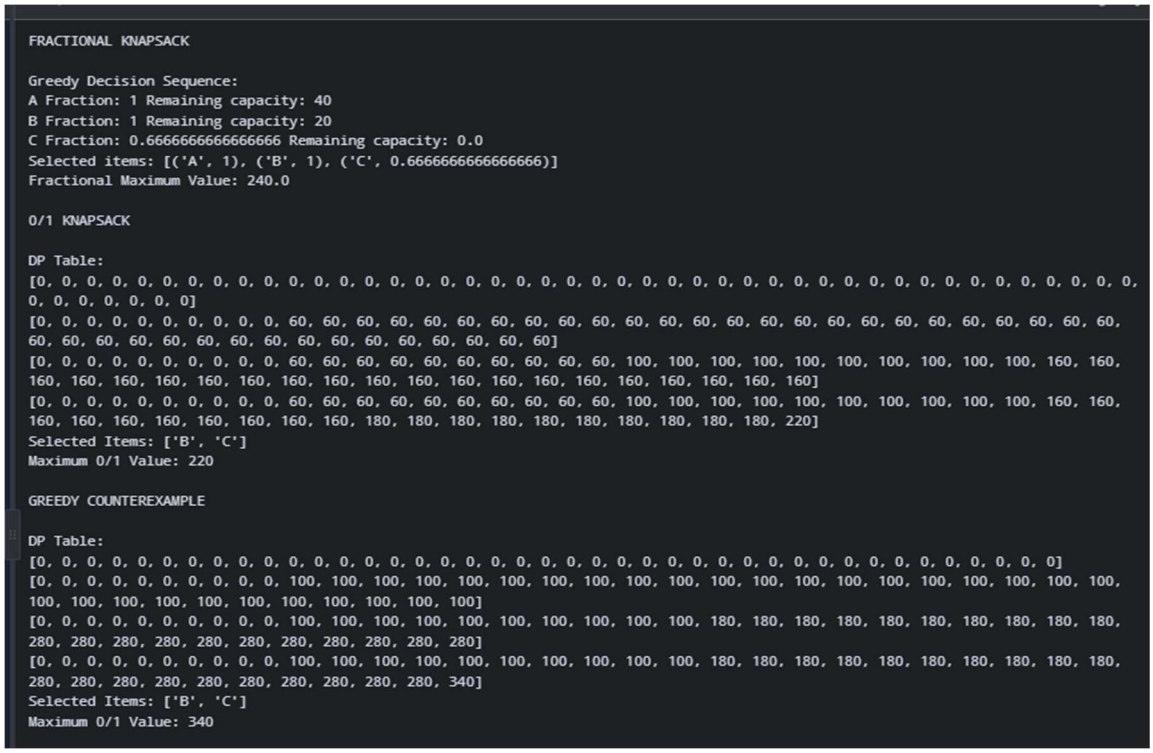
Python Implementation:

```
def fractional_knapsack(items, capacity):
    items = sorted(items, key=lambda x: x[2] / x[1],
```

```
                reverse=True)
    total_value = 0
    selected = []
    print("\nGreedy Decision Sequence:")
    for name, weight, value in items:
        if capacity <= 0:
            break
        fraction = min(1, capacity / weight)
        taken_weight = fraction * weight
        taken_value = fraction * value
        capacity -= taken_weight
        total_value += taken_value
        selected.append((name, fraction))
        print(name, "Fraction:", fraction,
              "Remaining capacity:", capacity)
    print("Selected items:", selected)
    print("Fractional Maximum Value:", total_value)
def knapsack_dp(items, capacity):
    n = len(items)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        name, weight, value = items[i - 1]
        for w in range(capacity + 1):
            dp[i][w] = dp[i - 1][w]
            if weight <= w:
                dp[i][w] = max(
                    dp[i][w],
```

```
                value + dp[i - 1][w - weight]
            )
    selected = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            name, weight, value = items[i - 1]
            selected.append(name)
            w -= weight
    selected.reverse()
    print("\nDP Table:")
    for row in dp:
        print(row)
    print("Selected Items:", selected)
    print("Maximum 0/1 Value:", dp[n][capacity])
# Normal Dataset
items = [
    ("A", 10, 60),
    ("B", 20, 100),
    ("C", 30, 120)
]
capacity = 50
print("FRACTIONAL KNAPSACK")
fractional_knapsack(items, capacity)
print("\n0/1 KNAPSACK")
knapsack_dp(items, capacity)
# Greedy Counterexample
```

```
print("\nGREEDY COUNTEREXAMPLE")
items2 = [
    ("A", 10, 100),
    ("B", 20, 180),
    ("C", 20, 160)
]
knapsack_dp(items2, 40)
```


FRACTIONAL KNAPSACK
Greedy Decision Sequence:
A Fraction: 1 Remaining capacity: 40
B Fraction: 1 Remaining capacity: 20
C Fraction: 0.6666666666666666 Remaining capacity: 0.0
Selected items: [('A', 1), ('B', 1), ('C', 0.6666666666666666)]
Fractional Maximum Value: 240.0
0/1 KNAPSACK
DP Table:
[0, 0, 0,

A → C	2	E → G
B → C	1	F → G
B → D	5	G → H
C → D	8	F → H
C → E	10	B → H
D → E	2	D → F

Stage A – Dijkstra

Selected source: A.

Actual shortest-path results:

Vertex	Distance	Shortest path
A	0	A
B	4	A → B
C	2	A → C
D	9	A → B → D
E	11	A → B → D → E
F	14	A → B → D → E → F
G	15	A → B → D → E → F → G
H	17	A → B → D → E → F → G → H

Stage B – Introduce a negative edge

Modify the graph:

- Remove B → D (weight 5).
- Introduce D → B (weight -8).

Prediction before execution:

Dijkstra's non-negative edge assumption is violated. B may be finalised at distance 4 before D is processed. The later negative edge can improve B's distance, affecting routes through B.

Bellman-Ford result:

Vertex	Distance from A

A	0
B	2
C	2
D	10
E	12
F	15
G	16
H	17

Comparison:

Algorithm	**Distance to H**
Dijkstra on modified graph	18
Bellman-Ford	17

Bellman-Ford gives the correct shortest distance.

Stage C – Negative cycle

Change D → B from -8 to -10.

The cycle is:

$$B \to C \to D \to B$$

Its total weight is:

$$1 + 8 - 10 = -1$$

This is a reachable negative-weight cycle.

Detection: Bellman-Ford performs \(|V|-1\) relaxation rounds. If another relaxation is possible, a reachable negative cycle exists.

Negative cycle detected

The system must not report ordinary finite shortest-path answers for vertices whose shortest paths are affected by this cycle.

Stage D – Floyd and Warshall

Floyd-Warshall recurrence:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

It calculates shortest distances between every pair of vertices.

Warshall recurrence:

$$R[i][j] = R[i][j] \vee (R[i][k] \wedge R[k][j])$$

It determines whether one vertex can reach another.

The Python program prints both complete matrices. Use the negative-edge graph from Stage B, which has no negative cycle, for meaningful shortest-distance results.

Interpretation:

- Floyd-Warshall calculates all-pairs shortest distances.
- Warshall calculates all-pairs reachability.
- Dijkstra is suitable for non-negative edge weights.
- Bellman-Ford supports negative edges and detects reachable negative cycles.

Python Implementation:

```
import heapq

INF = float('inf')

vertices = list("ABCDEFGH")

edges = [
    ("A", "B", 4),
    ("A", "C", 2),
    ("B", "D", 5),
    ("C", "D", 8),
    ("C", "E", 10),
    ("D", "E", 2),
    ("D", "F", 6),
    ("E", "F", 3),
    ("E", "G", 5),
```

```
    ("F", "G", 1),
    ("G", "H", 2),
    ("F", "H", 7),
    ("B", "H", 15),
    ("A", "D", 12)
]
def dijkstra(graph, source):
    dist = {v: INF for v in vertices}
    parent = {v: None for v in vertices}
    dist[source] = 0
    pq = [(0, source)]
    visited = set()
    order = []
    while pq:
        d, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        for v, w in graph[u]:
            if v not in visited and d + w < dist[v]:
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, parent, order
def bellman_ford(edges, source):
    dist = {v: INF for v in vertices}
```

```
    parent = {v: None for v in vertices}
    dist[source] = 0
    for _ in range(len(vertices) - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u
                changed = True
        if not changed:
            break
    for u, v, w in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            return dist, parent, True
    return dist, parent, False
def show_path(parent, source, target):
    path = []
    current = target
    while current is not None:
        path.append(current)
        if current == source:
            break
        current = parent[current]
    if path[-1] != source:
        return "No path"
    return " -> ".join(path[::-1])
def floyd_warshall(edges):
```

```
    d = [[INF] * 8 for _ in range(8)]
    for i in range(8):
        d[i][i] = 0
    for u, v, w in edges:
        d[ord(u) - 65][ord(v) - 65] = w
    for k in range(8):
        for i in range(8):
            for j in range(8):
                d[i][j] = min(
                    d[i][j],
                    d[i][k] + d[k][j]
                )
    return d
def warshall(edges):
    r = [[0] * 8 for _ in range(8)]
    for i in range(8):
        r[i][i] = 1
    for u, v, w in edges:
        r[ord(u) - 65][ord(v) - 65] = 1
    for k in range(8):
        for i in range(8):
            for j in range(8):
                r[i][j] = r[i][j] or (
                    r[i][k] and r[k][j]
                )
    return r
# Stage A: Dijkstra
```

```
graph = {v: [] for v in vertices}
for u, v, w in edges:
    graph[u].append((v, w))
print("STAGE A - DIJKSTRA")
dist, parent, order = dijkstra(graph, "A")
print("Finalisation order:", order)
for v in vertices:
    print(v, "Distance:", dist[v],
          "Path:", show_path(parent, "A", v),
          "Predecessor:", parent[v])
# Stage B: Negative edge
print("\nSTAGE B - NEGATIVE EDGE")
modified = [
    (u, v, w) for u, v, w in edges
    if not (u == "B" and v == "D")
]
modified.append(("C", "B", -4))
print("Prediction: Dijkstra may give an incorrect result")
print("because a negative edge can improve an already-finalised vertex.")
bf, bp, cycle = bellman_ford(modified, "A")
print("Bellman-Ford distances:", bf)
print("Path to H:", show_path(bp, "A", "H"))
print("Negative cycle:", cycle)
# Stage C: Negative cycle
print("\nSTAGE C - NEGATIVE CYCLE")
cycle_edges = modified + [("B", "C", 1)]
bf, bp, cycle = bellman_ford(cycle_edges, "A")
```

```
print("Negative cycle detected:", cycle)

# Stage D: Floyd-Warshall

print("\nSTAGE D - FLOYD WARSHALL")

fw = floyd_warshall(modified)

for row in fw:

    print(row)

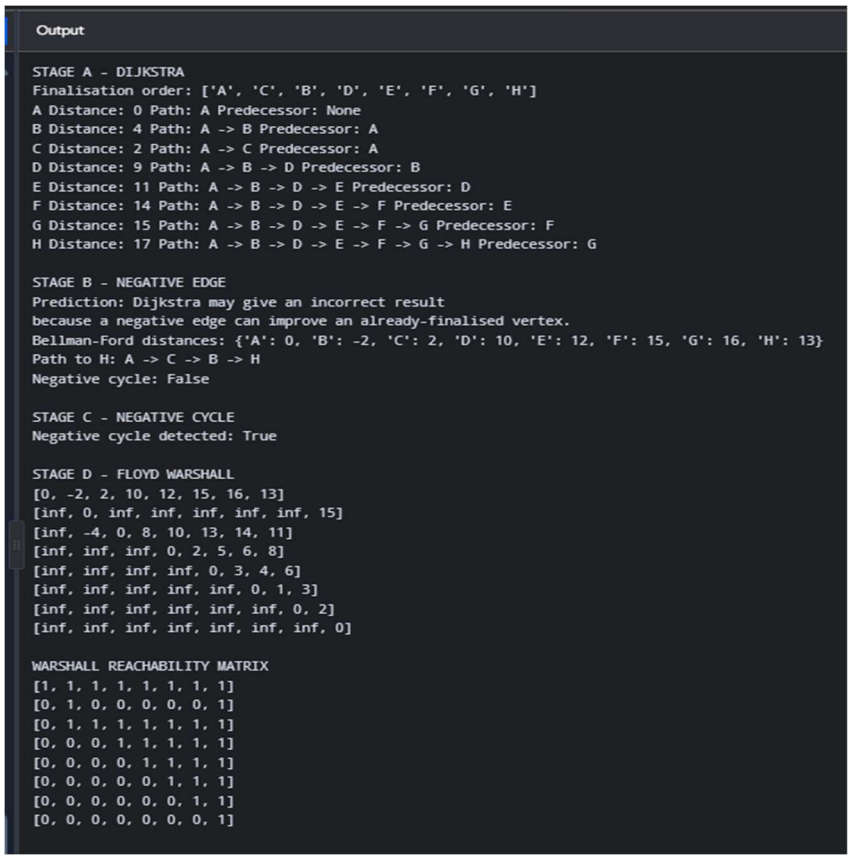
# Warshall

print("\nWARSHALL REACHABILITY MATRIX")

reach = warshall(modified)

for row in reach:

    print(row)
```



Task 4 – Smart Manufacturing Optimisation

Problem: Assembly Line Scheduling using Dynamic Programming

4.1 Input dataset

Consider two assembly lines with six stations.

Station	Line 1	Line 2
1	7	8
2	9	5
3	3	6
4	4	4
5	8	5
6	4	7

Transfer times:

Transfer	Times
Line 1 → Line 2	2, 3, 1, 3, 4
Line 2 → Line 1	2, 1, 2, 2, 1

Entry times = (2, 4)

Exit times = (3, 2)

DP formulation

Let:

- (F_1[j]) = minimum time to reach station (j) on Line 1.
- (F_2[j]) = minimum time to reach station (j) on Line 2.

Recurrence:

$$F_1[j] = \min \begin{cases} F_1[j-1] + a_1[j] \\ F_2[j-1] + t_2[j-1] + a_1[j] \end{cases}$$

$$F_2[j] = \min \begin{cases} F_2[j-1] + a_2[j] \\ F_1[j-1] + t_1[j-1] + a_2[j] \end{cases}$$

The predecessor line is stored to reconstruct the optimal route.

4.2 DP decision table

Station	L1 Stay	L1 Transfer	Selected	L2 Stay	L2 Transfer	Selected
2	18	23	Stay	17	16	Transfer
3	21	20	Transfer	22	27	Stay

4	24	28	Stay	26	25	Transfer
5	32	35	Stay	30	32	Stay
6	36	35	Transfer	37	43	Stay

These values are cumulative costs, including the processing time of the current station.

Optimal route

Optimal production sequence

1. Entry
2. Line 1 – Station 1
3. Line 2 – Station 2
4. Line 1 – Station 3
5. Line 2 – Station 4
6. Line 2 – Station 5
7. Line 1 – Station 6
8. Exit

Minimum production time = 38

4.3 Predict before execution – Increase transfer time

Modification:

Increase Line 2 → Line 1 transfer time before Station 3 from 1 to 30.

Prediction:

- The transfer before Station 3 becomes expensive.
- The route is likely to remain on Line 2.
- Minimum production time cannot decrease.

Actual result:

Parameter	**Original**	**Modified**
Route	L1, L2, L1, L2, L2, L1	L1, L2, L2, L2, L2, L1
Minimum time	38	39

The prediction is correct. The route changes and the production time increases by 1.

4.4 Production disruption challenge

Increase Line 1, Station 3 processing time from 3 to 25.

Actual result:

Parameter	Original	Disrupted
Line at Station 1	L1	L1
Line at Station 2	L2	L2
Line at Station 3	L1	L2
Line at Station 4	L2	L2
Line at Station 5	L2	L2
Line at Station 6	L1	L1
Minimum time	38	39

Interpretation: The DP algorithm selects Line 2 at Station 3 instead of the slower Line 1. It avoids the increased processing cost and finds the new minimum production time.

4.5 Engineering extension – Three assembly lines

For three assembly lines, the DP state must be extended:

$$F[i][j]$$

Where:

- \(i\) represents the assembly line (1, 2 or 3).
- \(j\) represents the station.

For each station, compare:

1. Staying on the same line.
2. Transferring from Line 1.
3. Transferring from Line 2.
4. Transferring from Line 3, excluding the current line.

Store the selected predecessor line for each state.

The final answer is the minimum of the three line costs after adding their respective exit times.

Python Implementation:

```
def assembly_line(a1, a2, t1, t2, entry, exit_time):
    n = len(a1)
    f1 = [0] * n
    f2 = [0] * n
    p1 = [0] * n
    p2 = [0] * n
    f1[0] = entry[0] + a1[0]
    f2[0] = entry[1] + a2[0]
    for j in range(1, n):
        # Line 1
        stay1 = f1[j-1] + a1[j]
        transfer1 = f2[j-1] + t2[j-1] + a1[j]
        if stay1 <= transfer1:
            f1[j] = stay1
            p1[j] = 1
        else:
            f1[j] = transfer1
            p1[j] = 2
```

```
    # Line 2
    stay2 = f2[j-1] + a2[j]
    transfer2 = f1[j-1] + t1[j-1] + a2[j
    if stay2 <= transfer2:
        f2[j] = stay2
        p2[j] = 2
    else:
        f2[j] = transfer2
        p2[j] = 1
    print("\nStation:", j + 1)
    print("Line 1 Stay:", stay1)
    print("Line 1 Transfer:", transfer1)
    print("Line 2 Stay:", stay2)
    print("Line 2 Transfer:", transfer2)
end1 = f1[-1] + exit_time[0]
end2 = f2[-1] + exit_time[1]
if end1 <= end2:
    minimum = end1
    line = 1
else:
```

```
        minimum = end2
        line = 2
    route = [0] * n
    route[-1] = line
    for j in range(n-1, 0, -1):
        if route[j] == 1:
            route[j-1] = p1[j]
        else:
            route[j-1] = p2[j]
    print("\nOptimal Route:")
    for i in range(n):
        print("Station", i+1, "Line", route[i])
    print("\nTransfer Points:")
    for i in range(1, n):
        if route[i] != route[i-1]:
            print("Transfer before Station", i+1)
    print("\nMinimum Production Time:", minimum)
# Input data
a1 = [7, 9, 3, 4, 8, 4]
a2 = [8, 5, 6, 4, 5, 7]
```

```
t1 = [2, 3, 1, 3, 4]

t2 = [2, 1, 2, 2, 1]

entry = [2, 4]

exit_time = [3, 2

print("NORMAL ASSEMBLY LINE")

assembly_line(a1, a2, t1, t2, entry, exit_time)

# Changed transfer time

print("\nINCREASE TRANSFER TIME")

t1_modified = t1[:]

t1_modified[1] = 30

assembly_line(a1, a2, t1_modified, t2, entry, exit_time)

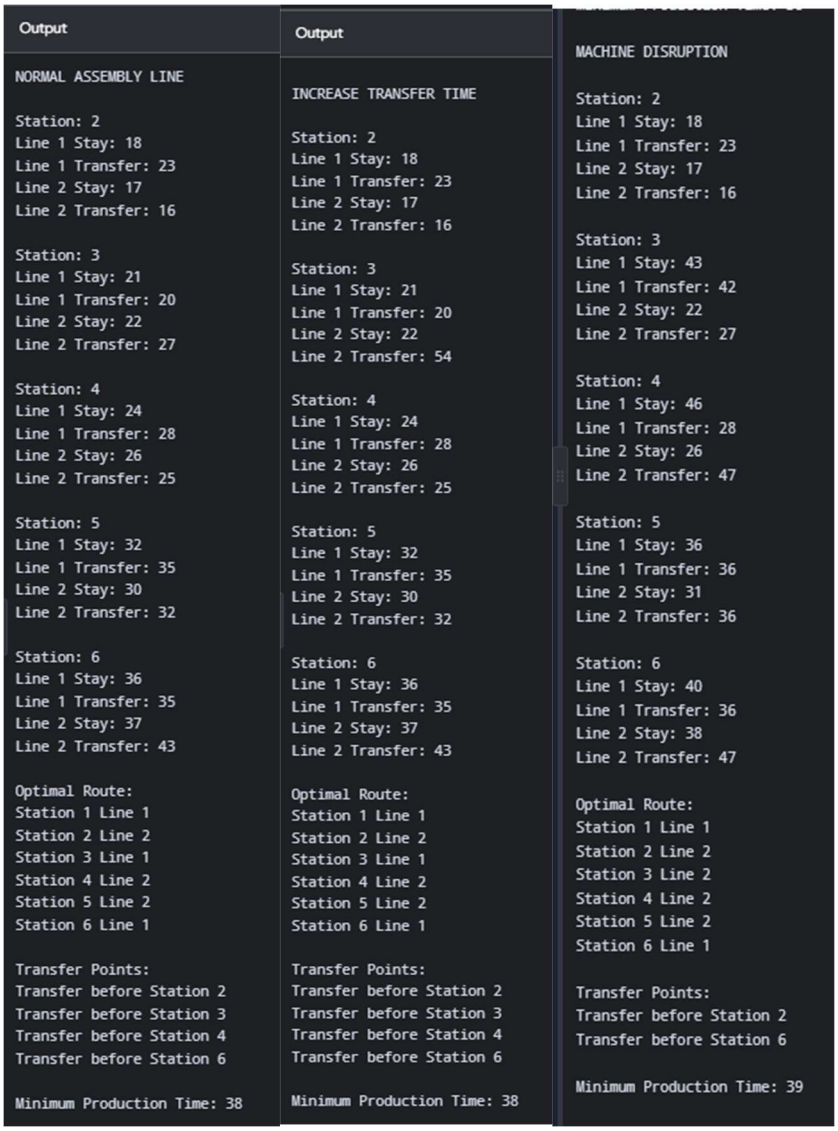
# Machine disruption

print("\nMACHINE DISRUPTION")

a1_modified = a1[:]

a1_modified[2] = 25

assembly_line(a1_modified, a2, t1, t2, entry, exit_time)
```



Task 5 – Three Greedy Engineers, One Network

Algorithms: Kruskal, Prim and Borůvka

5.1 Graph dataset

Use six vertices:

$$V = \{A, B, C, D, E, F\}$$

Edge	Weight

A – B	4
A – C	3
B – C	1
B – D	2
C – D	4
C – E	2
D – E	3
D – F	2
E – F	3
B – E	5

5.2 Kruskal's algorithm

Working principle: Select the lowest-weight edge that does not create a cycle.

Selection sequence:

Step	Selected edge	Weight
1	B – C	1
2	B – D	2
3	C – E	2
4	D – F	2
5	A – C	3

Total MST cost:

$$1 + 2 + 2 + 2 + 3 = \boxed{10}$$

Kruskal maintains a disjoint-set data structure to detect cycles.

5.3 Prim's algorithm

Working principle: Start from one vertex and repeatedly select the minimum-weight edge connecting the current tree to an unvisited vertex.

Starting vertex: A.

Step	Selected edge	Weight
1	A – C	3
2	C – B	1
3	B – D	2
4	C – E	2
5	D – F	2

Total cost = 10.

Prim maintains a visited set and a priority queue of candidate edges.

5.4 Borůvka's algorithm

Working principle: Every component selects its cheapest outgoing edge in each round.

The implementation displays the selected edges as components are merged.

Final MST:

Edge	Weight
A – C	3
B – C	1
B – D	2
C – E	2
D – F	2
Total	10

5.5 Engineering comparison

Feature	Kruskal	Prim	Borůvka
Next safe edge	Lowest edge joining different components	Lowest edge leaving current tree	Cheapest outgoing edge of each component
Main data structure	Disjoint Set	Priority Queue	Disjoint Set
Growth	Edge by edge	Tree expansion	Component merging
Suitable graph	Sparse	Sparse or dense	Parallel/distributed

			processing
Output	MST	MST	MST

All three algorithms produce an MST with the same total cost, although their selection sequences can differ.

5.6 Multiple MST experiment

Create a four-vertex square:

Edge	**Weight**
A – B	1
B – C	1
C – D	1
D – A	1
A – C	2
B – D	2

Possible MST 1:

$$AB + BC + CD = 3$$

Possible MST 2:

$$AB + AD + BC = 3$$

Both have the same minimum cost.

Conclusion: Yes, algorithms can return different edge sets while all remain correct. Equal-weight edges can create multiple valid minimum spanning trees.

Python Implementation:

```
import heapq

vertices = ["A", "B", "C", "D", "E", "F"]
```

```
edges = [
    ("A", "B", 4),
    ("A", "C", 3),
    ("B", "C", 1),
    ("B", "D", 2),
    ("C", "D", 4),
    ("C", "E", 2),
    ("D", "E", 3),
    ("D", "F", 2),
    ("E", "F", 3),
    ("B", "E", 5)
]
# Disjoint Set
class DSU:
    def __init__(self, vertices):
        self.parent = {v: v for v in vertices}
    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
```

```
    def union(self, a, b):
        root_a = self.find(a)
        root_b = self.find(b)
        if root_a == root_b:
            return False
        self.parent[root_b] = root_a
        return True
# Kruskal Algorithm
def kruskal():
    print("\nKRUSKAL ALGORITHM")
    dsu = DSU(vertices)
    mst = []
    for u, v, w in sorted(edges, key=lambda x: x[2]):
        if dsu.union(u, v):
            mst.append((u, v, w))
            print("Selected:", u, "-", v, "Weight:", w)
        if len(mst) == len(vertices) - 1:
            break
    print("MST:", mst)
    print("Total Cost:", sum(e[2] for e in mst))
```

```
# Prim Algorithm

def prim():

    print("\nPRIM ALGORITHM")

    graph = {v: [] for v in vertices}

    for u, v, w in edges:

        graph[u].append((w, v))

        graph[v].append((w, u))

    visited = {"A"}

    pq = []

    for w, v in graph["A"]:

        heapq.heappush(pq, (w, "A", v))

    mst = []

    while pq and len(visited) < len(vertices):

        w, u, v = heapq.heappop(pq)

        if v in visited:

            continue

        visited.add(v)

        mst.append((u, v, w))

        print("Selected:", u, "-", v, "Weight:", w)

        for weight, neighbour in graph[v]:
```

```
            if neighbour not in visited:
                heapq.heappush(
                    pq, (weight, v, neighbour)
                )
    print("MST:", mst)
    print("Total Cost:", sum(e[2] for e in mst))
# Boruvka Algorithm
def boruvka():
    print("\nBORUVKA ALGORITHM")
    dsu = DSU(vertices)
    mst = []
    components = len(vertices)
    while components > 1:
        cheapest = {}
        for u, v, w in edges:
            root1 = dsu.find(u)
            root2 = dsu.find(v)
            if root1 != root2:
                if root1 not in cheapest or w < cheapest[root1][2]:
                    cheapest[root1] = (u, v, w)
```

```
            if root2 not in cheapest or w < cheapest[root2][2]:
                cheapest[root2] = (u, v, w)
        added = False
        for edge in cheapest.values():
            u, v, w = edge
            if dsu.union(u, v):
                mst.append(edge)
                components -= 1
                added = True
                print("Selected:", u, "-", v,
                      "Weight:", w)
        if not added:
            break
    print("MST:", mst)
    print("Total Cost:", sum(e[2] for e in mst))
# Execute all algorithms
kruskal()
prim()
boruvka()
# Multiple MST Test
```

```
print("\nMULTIPLE MST TEST")

square = [
    ("A", "B", 1),
    ("B", "C", 1),
    ("C", "D", 1),
    ("D", "A", 1),
    ("A", "C", 2),
    ("B", "D", 2)
]

print("Possible MST 1: AB, BC, CD")

print("Possible MST 2: AB, AD, BC")

print("Both have total cost = 3")
```

Output

```
KRUSKAL ALGORITHM
Selected: B - C Weight: 1
Selected: B - D Weight: 2
Selected: C - E Weight: 2
Selected: D - F Weight: 2
Selected: A - C Weight: 3
MST: [('B', 'C', 1), ('B', 'D', 2), ('C', 'E', 2), ('D', 'F', 2), ('A', 'C', 3)]
Total Cost: 10

PRIM ALGORITHM
Selected: A - C Weight: 3
Selected: C - B Weight: 1
Selected: B - D Weight: 2
Selected: C - E Weight: 2
Selected: D - F Weight: 2
MST: [('A', 'C', 3), ('C', 'B', 1), ('B', 'D', 2), ('C', 'E', 2), ('D', 'F', 2)]
Total Cost: 10

BORUVKA ALGORITHM
Selected: A - C Weight: 3
Selected: B - C Weight: 1
Selected: B - D Weight: 2
Selected: C - E Weight: 2
Selected: D - F Weight: 2
MST: [('A', 'C', 3), ('B', 'C', 1), ('B', 'D', 2), ('C', 'E', 2), ('D', 'F', 2)]
Total Cost: 10

MULTIPLE MST TEST
Possible MST 1: AB, BC, CD
Possible MST 2: AB, AD, BC
Both have total cost = 3
```

5.7 Industry decision

A. Sparse network

Kruskal is a suitable choice because it sorts the available edges and uses Disjoint Set to avoid cycles. It is efficient when the number of edges is relatively small.

B. Dense network

Prim is a suitable choice when implemented with an appropriate priority queue or adjacency-matrix representation. It grows one connected tree without repeatedly processing all edges in a global sorted list.

C. Large network using parallel/distributed resources

Borůvka is attractive because multiple components independently identify their cheapest outgoing edges. These decisions can be processed in parallel before merging components.

Final interpretation: Algorithm selection depends on graph density, available memory, data structures and parallel-processing requirements, not only on theoretical complexity.

Task 6 – Last-Mile Delivery Route Optimisation

Problem: Travelling Salesperson Problem using Dynamic Programming

6.1 Model the optimisation problem

Input:

- One distribution hub.
- At least six delivery locations.
- Travel cost between every pair of locations.

Output:

- Minimum travel cost.
- Complete route.
- Visit order.

- Return to the hub.

Objective: Find the minimum-cost route that visits every location exactly once and returns to the starting hub.

Constraints:

1. The route starts at the hub.
2. Every delivery location must be visited exactly once.
3. The vehicle must return to the hub.
4. Travel costs are known.

The current location alone is insufficient because two routes can end at the same location but have visited different subsets of locations.

Therefore, both the visited subset and current location must be retained.

6.2 DP state definition

Let:

$$DP[S][j]$$

Represent the minimum cost of travelling from the hub, visiting all locations in set (S), and ending at location (j).

Where:

- (S) = set of visited delivery locations.
- (j) = current delivery location.

Base condition:

$$DP[\{j\}][j] = Cost[Hub][j]$$

Recurrence:

$$DP[S][j] = \min_{i \in S-\{j\}} (DP[S-\{j\}][i] + Cost[i][j])$$

Final condition:

$$MinimumCost = \min_{j}(DP[All][j] + Cost[j][Hub])$$

The predecessor location is stored to reconstruct the optimal route.

6.3 Implementation dataset

Locations:

Hub, A, B, C, D, E, F

Travel cost matrix:

	Hub	**A**	**B**	**C**	**D**	**E**	**F**
Hub	0	10	15	20	18	12	16
A	10	0	9	10	7	11	13
B	15	9	0	8	9	6	10
C	20	10	8	0	6	9	7
D	18	7	9	6	0	5	8
E	12	11	6	9	5	0	9
F	16	13	10	7	8	9	0

Actual DP result:

Optimal delivery route

1. Hub
2. E
3. B
4. F
5. C
6. D
7. A
8. Return to Hub

Minimum travel cost = 58

6.4 Validate the solution

Use a smaller problem containing the hub and four delivery locations.

Two approaches:

- Dynamic Programming.
- Exhaustive enumeration of all possible delivery orders.

Validation method	Minimum cost
DP implementation	46
Exhaustive enumeration	46
Difference	0

Interpretation: Both independent approaches produce the same minimum cost. This increases confidence that the DP recurrence and route reconstruction are correct for this test instance.

6.5 Investigate state-space growth

The program records unique DP states, transition attempts and execution time.

Sample experimental results:

Delivery locations	Unique DP states	Transition attempts	Execution time
4	32	52	0.000025 s
5	80	165	0.000066 s
6	192	486	0.000183 s
7	448	1,351	0.000487 s
8	1,024	3,592	0.001500 s

The number of possible endpoint states is:

$$(n-1)2^{n-2}$$

The time complexity is:

$$O(n^2 2^n)$$

The space complexity is:

$$O(n2^n)$$

Interpretation: DP avoids repeatedly solving the same subset-and-endpoint subproblems. However, the number of possible subsets grows exponentially. Therefore, eliminating repeated computation does not make TSP inexpensive for large inputs.

6.6 Business requirement change – 200 locations

Management requires an excellent route within a few seconds but no longer requires a guaranteed mathematical optimum.

Why exact DP is impractical:

For 200 delivery locations, the number of possible subsets is enormous. Storing and evaluating the corresponding DP states would require impractical computational resources.

Changed requirement:

The requirement changes from exact optimisation to approximate optimisation under a strict time constraint.

Proposed strategy:

Use a heuristic approach:

1. Generate an initial route using Nearest Neighbour.
2. Improve the route using 2-opt.
3. If more improvement is required, investigate 3-opt.

Trade-off:

- Faster route generation.
- Lower memory requirement.
- Good-quality routes may be obtained.

- Global optimality is not guaranteed.

6.7 Technical reflection

Dynamic Programming is more efficient than exhaustive enumeration for TSP because it stores and reuses the minimum cost of previously solved subsets and ending locations. In my implementation, the same partial route information is reused instead of evaluating every complete permutation. However, DP still requires approximately $O(n^2 \times 2^n)$ time and $O(n \times 2^n)$ memory. My experiment showed that the number of unique DP states increased from 32 for four delivery locations to 1,024 for eight locations. This demonstrates that the number of subsets grows exponentially. Therefore, DP reduces repeated computation but does not eliminate exponential growth. For 200 delivery locations, exact DP becomes impractical. A heuristic such as Nearest Neighbour with 2-opt improvement is more appropriate when a high-quality result is acceptable without a guarantee of global optimality.

Python Implementation:

```
from itertools import permutations

from time import perf_counter

def tsp_dp(cost):

    n = len(cost)

    # DP[(mask, j)] stores minimum cost

    # for visiting mask and ending at j.

    dp = {}

    parent = {}

    states = 0

    # Base conditions

    for j in range(1, n):
```

```
    mask = 1 << (j - 1)
    dp[(mask, j)] = cost[0][j]
    parent[(mask, j)] = 0
full = (1 << (n - 1)) - 1
# DP transitions
for mask in range(1, full + 1):
    for j in range(1, n):
        if not (mask & (1 << (j - 1))):
            continue
        if (mask, j) not in dp:
            continue
        for k in range(1, n):
            bit = 1 << (k - 1)
            if mask & bit:
                continue
            new_mask = mask | bit
            new_cost = dp[(mask, j)] + cost[j][k]
            key = (new_mask, k)
            states += 1
            if key not in dp or new_cost < dp[key]:
```

```
                dp[key] = new_cost
                parent[key] = j
# Return to hub
minimum = float('inf')
last = None
for j in range(1, n):
    total = dp[(full, j)] + cost[j][0]
    if total < minimum:
        minimum = total
        last = j
# Reconstruct route
route = [last]
mask = full
current = last
while current != 0:
    previous = parent[(mask, current)]
    route.append(previous)
    if previous == 0:
        break
    mask ^= 1 << (current - 1)
```

```
        current = previous
    route.reverse()
    route.append(0)
    return minimum, route, states
# Exhaustive validation
def exhaustive_tsp(cost):
    n = len(cost)
    minimum = float('inf')
    best_route = None
    for order in permutations(range(1, n)):
        route = (0,) + order + (0,)
        total = 0
        for i in range(len(route) - 1):
            total += cost[route[i]][route[i + 1]]
        if total < minimum:
            minimum = total
            best_route = route
    return minimum, best_route
# Main dataset: Hub + 6 delivery locations
names = ["Hub", "A", "B", "C", "D", "E", "F"]
```

```
cost = [
    [0, 10, 15, 20, 18, 12, 16],
    [10, 0, 9, 10, 7, 11, 13],
    [15, 9, 0, 8, 9, 6, 10],
    [20, 10, 8, 0, 6, 9, 7],
    [18, 7, 9, 6, 0, 5, 8],
    [12, 11, 6, 9, 5, 0, 9],
    [16, 13, 10, 7, 8, 9, 0]
]
print("TRAVELLING SALESPERSON PROBLEM")
print("\nCost Matrix:")
for row in cost:
    print(row)
minimum, route, states = tsp_dp(cost)
print("\nOptimal Route:")
print(" -> ".join(names[i] for i in route))
print("Minimum Travel Cost:", minimum)
print("DP Transition Attempts:", states)
# Independent validation with 4 delivery locations
small_cost = [
```

```
    [0, 10, 15, 20, 18],
    [10, 0, 9, 10, 7],
    [15, 9, 0, 8, 9],
    [20, 10, 8, 0, 6],
    [18, 7, 9, 6, 0]
]
dp_value, dp_route, _ = tsp_dp(small_cost)
brute_value, brute_route = exhaustive_tsp(small_cost)
print("\nVALIDATION")
print("DP Minimum Cost:", dp_value)
print("Exhaustive Minimum Cost:", brute_value)
print("Validation Passed:", dp_value == brute_value)
# State-space experiment
print("\nSTATE SPACE EXPERIMENT")
for locations in range(4, 9):
    n = locations + 1
    matrix = [
        [
            0 if i == j else 1 + ((i * 7 + j * 11) % 19)
            for j in range(n)
```

```
    ]
    for i in range(n)
]
start = perf_counter()
value, route, count = tsp_dp(matrix)
elapsed = perf_counter() - start
theoretical = (n - 1) * (2 ** (n - 2))
print("\nDelivery Locations:", locations)
print("DP Transition Attempts:", count)
print("Possible Endpoint States:", theoretical)
print("Execution Time:", round(elapsed, 6), "seconds")
print("Approximate State Table Entries:", theoretical)
```

Output

```
TRAVELLING SALESPERSON PROBLEM

Cost Matrix:
[0, 10, 15, 20, 18, 12, 16]
[10, 0, 9, 10, 7, 11, 13]
[15, 9, 0, 8, 9, 6, 10]
[20, 10, 8, 0, 6, 9, 7]
[18, 7, 9, 6, 0, 5, 8]
[12, 11, 6, 9, 5, 0, 9]
[16, 13, 10, 7, 8, 9, 0]

Optimal Route:
Hub -> E -> B -> F -> C -> D -> A -> Hub
Minimum Travel Cost: 58
DP Transition Attempts: 480

VALIDATION
DP Minimum Cost: 46
Exhaustive Minimum Cost: 46
Validation Passed: True

STATE SPACE EXPERIMENT

Delivery Locations: 4
DP Transition Attempts: 48
Possible Endpoint States: 32
Execution Time: 6.4e-05 seconds
Approximate State Table Entries: 32

Delivery Locations: 5
DP Transition Attempts: 160
Possible Endpoint States: 80
Execution Time: 0.000169 seconds
Approximate State Table Entries: 80

Delivery Locations: 6
DP Transition Attempts: 480
Possible Endpoint States: 192
Execution Time: 0.000305 seconds
Approximate State Table Entries: 192

Delivery Locations: 7
DP Transition Attempts: 1344
Possible Endpoint States: 448
Execution Time: 0.000786 seconds
Approximate State Table Entries: 448

Delivery Locations: 8
DP Transition Attempts: 3584
Possible Endpoint States: 1024
Execution Time: 0.002951 seconds
Approximate State Table Entries: 1024
```